

图像频域水印项目使用手册

1. 项目功能简介

本项目用于完成“题目5：图像频域水印”大作业，覆盖基础、进阶和挑战三个层次的要求。

1.1 基础任务

- 对图像进行 DFT 等频域变换。
- 在频域特定位置嵌入不可见数字水印。
- 通过逆变换得到含水印图像。
- 从含水印图像中提取原始水印。
- 使用 PSNR、SSIM、MSE 等指标验证水印不可见性。

1.2 进阶任务

- 分析不同嵌入强度对图像质量的影响。
- 测试 JPEG 压缩、缩放、加噪后水印是否仍能提取。

1.3 挑战任务

- 实现 DCT 域水印。
- 实现 DWT 域水印。
- 实现 DCT 盲水印算法，提取时不需要原始图像。
- 增加彩色图像 Y 通道频域水印接口，使彩色图像输出仍保持彩色。

2. 项目目录说明

```
frequency_domain_watermarking/
├── configs/
│   └── experiments.yaml          # 实验配置文件
├── data/
│   ├── images/                  # BSD 彩色图像数据
│   │   ├── train/
│   │   └── test/
│   ├── misc/                    # USC-SIPI 图像，含灰度和彩色图
│   ├── raw/                     # 默认原始图像目录
│   └── watermark/
│       └── logo.png             # 当前 HJQ 水印图
├── experiments/
│   ├── common.py                # 实验公共工具
│   └── run_basic.py             # 基础嵌入/提取实验
```

```
|   |   | run_strength.py          # 嵌入强度实验
|   |   | run_attacks.py         # 抗攻击实验
|   |   | run_all.py             # 一键运行全部实验
|   |   |
|   |   | outputs/
|   |   |   | multi_image/       # 多图实验输出结果
|   |   |
|   |   | src/
|   |   |   | watermarking/
|   |   |       | algorithms/
|   |   |       |   | dft.py      # DFT 水印算法
|   |   |       |   | dct.py      # DCT 盲水印算法
|   |   |       |   | dwf.py      # DWT 水印算法
|   |   |       | attacks.py      # 攻击模拟
|   |   |       | io_utils.py     # 图像读写、Y 通道处理
|   |   |       | metrics.py      # 指标计算
|   |   |       | registry.py     # 算法注册表
|   |   |       | visualization.py # 可视化输出
|   |   |
|   |   | tests/                 # 自动化测试
|   |   |
|   |   | main.py                # 命令行接口
|   |   |
|   |   | README.md
|   |   | requirements.txt
|   |   | pyproject.toml
```

3. 第一次运行前准备

3.1 进入项目目录

打开终端，进入项目根目录：

```
cd /Users/mercury/Desktop/Research/frequency_domain_watermarking
```

确认当前目录：

```
pwd
```

应看到类似：

```
/Users/mercury/Desktop/Research/frequency_domain_watermarking
```

3.2 安装依赖

运行：

```
python3 -m pip install -r requirements.txt
```

如果看到很多：

```
Requirement already satisfied
```

说明依赖已经安装过，可以继续运行。

主要依赖如下：

依赖	作用
numpy	数值计算、矩阵处理
opencv-python	图像读取、保存、DCT、颜色空间转换
scipy	科学计算辅助
scikit-image	SSIM 等图像质量指标
matplotlib	画图、保存曲线
PyWavelets	DWT 小波变换
PyYAML	读取 YAML 配置文件
pandas	保存 CSV 实验数据
requests	自动下载示例图像
pytest	自动化测试

4. 验证项目是否能正常运行

建议先运行自动化测试：

```
python3 -m pytest -v
```

如果成功，会看到：

```
26 passed, 1 warning
```

其中 `warning` 是 macOS 自带 LibreSSL 与 urllib3 的兼容提醒，不影响实验运行。

5. 一键运行全部实验

运行：

```
PYTHONPATH=.:src python3 experiments/run_all.py
```

该命令会依次运行：

1. 基础实验： `run_basic.py`
2. 嵌入强度实验： `run_strength.py`
3. 抗攻击实验： `run_attacks.py`

运行完成后，结果保存在：

```
outputs/multi_image/
```

可以直接用 Finder 打开：

```
open outputs/multi_image
```

6. 为什么命令前要写 PYTHONPATH=.:src

命令前面的：

```
PYTHONPATH=.:src
```

是为了让 Python 能找到项目里的模块。

路径	作用
<code>.</code>	让 Python 找到 <code>experiments</code> 目录
<code>src</code>	让 Python 找到 <code>watermarking</code> 包

因此推荐每次运行实验脚本都使用：

```
PYTHONPATH=.:src python3 ...
```

7. 分别运行不同实验

7.1 基础实验

```
PYTHONPATH=.:src python3 experiments/run_basic.py
```

输出位置：

```
outputs/multi_image/basic/
```

该实验会对多张图像分别进行 DFT、DCT、DWT 嵌入与提取。

每张图像会生成一个单独目录，例如：

```
outputs/multi_image/basic/boat_gray/  
outputs/multi_image/basic/house_color/  
outputs/multi_image/basic/bsd_color/
```

每个目录中包含：

```
dft_watermarked.png  
dft_extracted.png  
dft_comparison.png  
dft_spectrum.png  
  
dct_watermarked.png  
dct_extracted.png  
dct_comparison.png  
dct_spectrum.png  
  
dwt_watermarked.png  
dwt_extracted.png  
dwt_comparison.png  
dwt_spectrum.png  
  
metrics_basic.csv
```

汇总数据：

```
outputs/multi_image/basic/metrics_basic_all.csv
```

7.2 嵌入强度实验

```
PYTHONPATH=.:src python3 experiments/run_strength.py
```

输出位置：

```
outputs/multi_image/strength/
```

该实验测试不同嵌入强度对图像质量和水印提取效果的影响。

典型输出：

```
metrics_strength.csv
strength_psnr.png
strength_ber.png
```

汇总数据：

```
outputs/multi_image/strength/metrics_strength_all.csv
```

7.3 抗攻击实验

```
PYTHONPATH=.:src python3 experiments/run_attacks.py
```

输出位置：

```
outputs/multi_image/attacks/
```

该实验测试：

1. JPEG 压缩攻击
2. 高斯噪声攻击
3. 缩放攻击

典型攻击图像文件：

```
dft_jpeg_90_attacked.png
dft_jpeg_70_attacked.png
dft_jpeg_50_attacked.png
dft_jpeg_30_attacked.png

dft_noise_5_attacked.png
dft_noise_10_attacked.png
dft_noise_20_attacked.png

dft_resize_0.5_attacked.png
dft_resize_0.75_attacked.png
dft_resize_1.5_attacked.png
```

汇总数据：

```
outputs/multi_image/attacks/metrics_attacks_all.csv
```

8. 当前默认使用的测试图片

默认配置在：

```
configs/experiments.yaml
```

当前使用 10 张图，包含灰度和彩色。

8.1 灰度图

```
boat_gray  
gray21_gray  
texture_gray  
pattern_gray
```

对应文件：

```
data/misc/boat.512.tiff  
data/misc/gray21.512.tiff  
data/misc/5.1.09.tiff  
data/misc/5.2.08.tiff
```

8.2 彩色图

```
house_color  
airplane_color  
bsd_color  
bsd_landscape_color  
bsd_train_color  
bsd_texture_color
```

对应文件：

```
data/misc/house.tiff  
data/misc/4.2.03.tiff  
data/images/test/101085.jpg  
data/images/test/103070.jpg  
data/images/train/100075.jpg  
data/images/train/189003.jpg
```

9. 灰度图和彩色图的处理方式

9.1 灰度图

灰度图只有一个亮度通道，算法直接对二维灰度矩阵做频域水印。

流程：

灰度图像

- DFT / DCT / DWT 变换
- 嵌入水印
- 逆变换
- 含水印灰度图

9.2 彩色图

彩色图一般有 RGB 三个通道。项目采用 Y 通道水印方案。

流程：

RGB 彩色图

- 转换到 YCrCb 空间
- 取出 Y 亮度通道
- 在 Y 通道上做 DFT / DCT / DWT 水印
- 将修改后的 Y 通道与原 Cr/Cb 色度通道合并
- 转回 RGB 彩色图

优点：

1. 保留彩色图像外观。
2. 水印主要嵌入亮度信息中。
3. 比直接对 R/G/B 三个通道分别嵌入更稳定。
4. 更适合作为报告中的扩展设计。

10. 当前水印

水印文件：

data/watermark/logo.png

当前内容为：

HJQ

查看水印：

open data/watermark/logo.png

11. 命令行单独嵌入和提取水印

11.1 DFT 灰度图嵌入

```
PYTHONPATH=.:src python3 main.py embed \  
--method dft \  
--image data/misc/boat.512.tiff \  
--watermark data/watermark/logo.png \  
--output outputs/demo/boat_dft_watermarked.png
```

提取：

```
PYTHONPATH=.:src python3 main.py extract \  
--method dft \  
--image outputs/demo/boat_dft_watermarked.png \  
--original data/misc/boat.512.tiff \  
--output outputs/demo/boat_dft_extracted.png
```

DFT 是非盲水印算法，提取时必须提供原图：

```
--original data/misc/boat.512.tiff
```

11.2 DCT 盲水印嵌入

```
PYTHONPATH=.:src python3 main.py embed \  
--method dct \  
--image data/misc/boat.512.tiff \  
--watermark data/watermark/logo.png \  
--output outputs/demo/boat_dct_watermarked.png
```

提取：

```
PYTHONPATH=.:src python3 main.py extract \  
--method dct \  
--image outputs/demo/boat_dct_watermarked.png \  
--output outputs/demo/boat_dct_extracted.png
```

DCT 是盲水印算法，不需要 `--original`。

11.3 DWT 水印嵌入

```
PYTHONPATH=.:src python3 main.py embed \  
--method dwt \  
--image data/misc/boat.512.tiff \  

```

```
--watermark data/watermark/logo.png \  
--output outputs/demo/boat_dwt_watermarked.png
```

提取：

```
PYTHONPATH=.:src python3 main.py extract \  
--method dwt \  
--image outputs/demo/boat_dwt_watermarked.png \  
--output outputs/demo/boat_dwt_extracted.png
```

11.4 彩色图 Y 通道嵌入

彩色图建议加 `--color-y`。

```
PYTHONPATH=.:src python3 main.py embed \  
--method dct \  
--color-y \  
--image data/images/test/101085.jpg \  
--watermark data/watermark/logo.png \  
--output outputs/demo/bsd_color_y_dct_watermarked.png
```

提取：

```
PYTHONPATH=.:src python3 main.py extract \  
--method dct \  
--color-y \  
--image outputs/demo/bsd_color_y_dct_watermarked.png \  
--output outputs/demo/bsd_color_y_dct_extracted.png
```

输出的 `bsd_color_y_dct_watermarked.png` 会保持彩色。

12. 单独运行攻击命令

12.1 JPEG 压缩攻击

```
PYTHONPATH=.:src python3 main.py attack \  
--type jpeg \  
--quality 50 \  
--image outputs/demo/boat_dft_watermarked.png \  
--output outputs/demo/boat_dft_jpeg50.png
```

`--quality` 表示 JPEG 质量因子，越低压缩越强。

值	说明
90	轻度压缩
70	中等压缩
50	较强压缩
30	强压缩

12.2 缩放攻击

```
PYTHONPATH=.:src python3 main.py attack \  
  --type resize \  
  --scale 0.5 \  
  --image outputs/demo/boat_dft_watermarked.png \  
  --output outputs/demo/boat_dft_resize05.png
```

`--scale 0.5` 表示先缩小到 0.5 倍，再恢复到原尺寸。

12.3 高斯噪声攻击

```
PYTHONPATH=.:src python3 main.py attack \  
  --type noise \  
  --sigma 10 \  
  --image outputs/demo/boat_dft_watermarked.png \  
  --output outputs/demo/boat_dft_noise10.png
```

`--sigma` 表示噪声标准差，越大噪声越强。

13. 配置文件说明

核心配置文件：

```
configs/experiments.yaml
```

主要分为三部分：

```
dataset:  
  ...  
methods:  
  ...  
experiments:  
  ...
```

14. dataset 参数说明

示例：

```
dataset:
  image: data/misc/boat.512.tiff
  watermark: data/watermark/logo.png
  image_size: [512, 512]
  images:
    - name: boat_gray
      path: data/misc/boat.512.tiff
      source_type: gray
    - name: house_color
      path: data/misc/house.tiff
      source_type: color
```

字段	含义
image	默认单图路径，用于兼容旧逻辑
watermark	水印图片路径
image_size	默认图像尺寸参考值
images	多图实验使用的图片列表

images 中每个图片有三个字段：

字段	含义
name	输出目录名和指标中的图像名
path	图片路径
source_type	图片类型，gray 或 color

添加彩色图片示例：

```
- name: my_image_color
  path: data/images/test/108005.jpg
  source_type: color
```

添加灰度图片示例：

```
- name: my_image_gray
  path: data/misc/5.1.10.tiff
  source_type: gray
```

15. methods 参数说明

```
methods:
  dft:
    alpha: 10.0
    watermark_size: [64, 64]
    radius_ratio: 0.28
  dct:
    delta: 12.0
    block_size: 8
    watermark_size: [32, 32]
    coeff_pair: [[3, 4], [4, 3]]
  dwt:
    alpha: 0.05
    wavelet: haar
    subband: hl
    watermark_size: [64, 64]
```

15.1 DFT 参数

参数	含义
alpha	DFT 水印嵌入强度
watermark_size	DFT 水印尺寸
radius_ratio	频谱嵌入区域半径比例

alpha 越大，水印越强、越容易提取，但图像失真也可能更明显。

radius_ratio 控制在频谱中心附近多大范围选择嵌入位置。中频区域通常能在不可见性和鲁棒性之间取得平衡。

15.2 DCT 参数

参数	含义
delta	DCT 系数差值控制参数
block_size	图像分块大小
watermark_size	DCT 水印尺寸
coeff_pair	用于表示 bit 的两个中频系数位置

DCT 盲水印通过两个中频系数大小关系表示 bit:

```
bit = 1: c1 > c2
bit = 0: c1 < c2
```

delta 越大，提取越稳定，但图像失真可能越明显。

15.3 DWT 参数

参数	含义
alpha	DWT 子带嵌入强度
wavelet	小波基函数
subband	嵌入水印的子带
watermark_size	DWT 水印尺寸

DWT 子带说明：

子带	含义
LL	低频近似部分
LH	水平方向细节
HL	垂直方向细节
HH	高频对角细节

当前默认使用 HL 子带。

16. experiments 参数说明

```
experiments:
  methods: [dft, dct, dwt]
  strengths:
    dft: [2, 5, 10, 20, 40]
    dct: [2, 5, 10, 20, 40]
    dwt: [0.01, 0.03, 0.05, 0.1]
  jpeg_qualities: [90, 70, 50, 30]
  noise_sigmas: [5, 10, 20]
  scale_factors: [0.5, 0.75, 1.5]
```

参数	含义
methods	要运行的算法列表

参数	含义
strengths	强度实验中的参数扫描范围
jpeg_qualities	JPEG 压缩质量因子
noise_sigmas	高斯噪声标准差
scale_factors	缩放攻击比例

17. 输出 CSV 指标说明

实验会输出：

```
metrics_basic_all.csv
metrics_strength_all.csv
metrics_attacks_all.csv
```

常见字段如下。

字段	含义
image_name	图像名称
image_path	图像原始路径
source_type	图像类型，gray 或 color
processing_channel	处理通道，gray 或 Y
method	算法名称，dft 、 dct 、 dwt
mse	均方误差，越小越好
psnr	峰值信噪比，越大越好
ssim	结构相似性，越接近 1 越好
nc	水印归一化相关系数，越接近 1 越好
ber	比特错误率，越小越好

17.1 MSE

MSE 表示原图和含水印图之间的平均平方差。越小表示图像越接近。

17.2 PSNR

PSNR 表示峰值信噪比，单位通常为 dB。

PSNR	图像质量
> 40 dB	失真极小
30—40 dB	质量较好
20—30 dB	有明显失真
< 20 dB	失真较大

17.3 SSIM

SSIM 表示结构相似性，通常范围为 0 到 1，越接近 1 表示越相似。

17.4 NC

NC 表示提取水印和原始水印的相关程度。越接近 1，表示水印提取越准确。

17.5 BER

BER 表示比特错误率，越小越好。

BER	说明
0	完全正确
0.01	约 1% bit 错误
0.1	约 10% bit 错误
0.5	接近随机猜测

18. 强度实验结果怎么看

强度实验输出：

```
outputs/multi_image/strength/metrics_strength_all.csv
outputs/multi_image/strength/strength_psnr_all.png
outputs/multi_image/strength/strength_ber_all.png
```

重点观察：

- 1. strength 增大时，psnr 是否下降。
- 2. strength 增大时，ber 是否下降。
- 3. 找到图像质量和提取准确率之间的平衡点。

理论趋势通常为：

嵌入强度增大

- 水印信号更强
- 提取更准确
- BER 降低
- 图像扰动增加
- PSNR 下降

19. 抗攻击实验结果怎么看

抗攻击实验输出：

```
outputs/multi_image/attacks/metrics_attacks_all.csv
outputs/multi_image/attacks/attack_ber_all.png
```

重点观察：

1. JPEG 质量越低，BER 是否升高。
2. 噪声 sigma 越大，BER 是否升高。
3. 缩放比例变化后，水印是否还能恢复。
4. DFT、DCT、DWT 哪个算法更稳定。

20. 基础实验结果怎么看

基础实验输出：

```
outputs/multi_image/basic/
```

每张图一个目录。最重要的是：

```
dft_comparison.png
dct_comparison.png
dwt_comparison.png
```

对比图通常包含：

Original	原始图像
Watermarked	含水印图像
Difference	差异图
Watermark	原始水印
Extracted	提取水印

看图时重点关注：

1. Original 和 Watermarked 是否肉眼接近。
2. Difference 是否整体较弱。
3. Extracted 是否能看出 HJQ 。
4. 彩色图输出是否仍保留颜色。

21. 三种算法解释

21.1 DFT 算法

DFT 是基础非盲算法，对整幅图像做二维傅里叶变换。

原图

- DFT
- 频谱中频区域嵌入水印
- IDFT
- 含水印图

优点：符合作业基础要求，能体现频域水印思想。

缺点：提取时需要原始图像。

21.2 DCT 算法

DCT 是挑战部分的主要盲水印算法。

图像分成 8×8 块

- 每块做 DCT
- 修改两个中频系数大小关系
- 每块表示一个 bit
- 逆 DCT 得到含水印图

提取时只需要含水印图，不需要原图。

21.3 DWT 算法

DWT 是小波域扩展算法。

原图

- DWT 分解
- 得到 LL / LH / HL / HH 子带
- 在指定子带中嵌入水印
- IDWT 重建图像

适合作为多分辨率频域方法展示。

22. 可调参数总结

参数位置	参数	可调内容
<code>dataset.images</code>	图片列表	增加、删除测试图片
<code>dataset.watermark</code>	水印路径	更换水印图片
<code>methods.dft.alpha</code>	DFT 强度	调整不可见性和鲁棒性
<code>methods.dct.delta</code>	DCT 强度	调整盲水印稳定性
<code>methods.dwt.alpha</code>	DWT 强度	调整小波域水印强度
<code>methods.dwt.subband</code>	DWT 子带	可选 <code>lh</code> 、 <code>hl</code> 、 <code>hh</code>
<code>experiments.methods</code>	算法列表	控制运行哪些算法
<code>experiments.strengths</code>	强度扫描	控制强度实验参数
<code>experiments.jpeg_qualities</code>	JPEG 质量	控制 JPEG 攻击强度
<code>experiments.noise_sigmas</code>	噪声强度	控制加噪攻击强度
<code>experiments.scale_factors</code>	缩放比例	控制缩放攻击强度

23. 推荐完整运行流程

如果从头完整运行一次，建议按以下顺序：

```
cd /Users/mercury/Desktop/Research/frequency_domain_watermarking
python3 -m pip install -r requirements.txt
python3 -m pytest -v
PYTHONPATH=./src python3 experiments/run_all.py
open outputs/multi_image
```

24. 当前实验数据规模

当前实验使用：

```
4 张灰度图
6 张彩色图
```

输出数据规模：

基础实验：30 行
强度实验：140 行
抗攻击实验：300 行

对应汇总文件：

```
outputs/multi_image/basic/metrics_basic_all.csv  
outputs/multi_image/strength/metrics_strength_all.csv  
outputs/multi_image/attacks/metrics_attacks_all.csv
```

25. 报告中可直接使用的描述

本实验选取 USC-SIPI 与 BSD 数据集中的多张灰度和彩色图像作为测试样本。对于灰度图像，直接在其二维灰度矩阵上进行 DFT、DCT 和 DWT 频域水印嵌入；对于彩色图像，先转换到 YCrCb 颜色空间，仅在亮度通道 Y 上进行水印嵌入，再与原始色度通道合成为彩色含水印图像。该方法既保留了彩色图像的视觉外观，又能利用频域方法完成水印嵌入与提取。

实验分别从不可见性和鲁棒性两个角度评价水印算法。不可见性通过 MSE、PSNR 和 SSIM 衡量；水印恢复质量通过 NC 和 BER 衡量。PSNR 和 SSIM 越高，表示含水印图像与原图越接近；NC 越高、BER 越低，表示提取水印与原始水印越一致。